

# 高级语言程序设计

## 实验报告

南开大学 计算机科学与技术

姓名 尹雯婕

学号 2512621

班级 计算机卓越拔尖计划伯苓 2 班

2026 年 5 月 13 日

## 目录

|                       |    |
|-----------------------|----|
| 一、作业题目 .....          | 3  |
| 二、开发与运行环境 .....       | 3  |
| 三、课题要求 .....          | 3  |
| 四、项目总体设计 .....        | 3  |
| 1. 整体流程 .....         | 3  |
| 2. 类与模块划分 .....       | 3  |
| 3. 游戏循环设计 .....       | 4  |
| 五、主要模块设计与实现 .....     | 4  |
| 1. 开始界面与说明界面 .....    | 4  |
| 2. 地图与路径设计 .....      | 5  |
| 3. 敌人系统 .....         | 5  |
| 4. 防御塔系统 .....        | 5  |
| 5. 子弹系统 .....         | 5  |
| 6. 胜负判断与结果界面 .....    | 5  |
| 7. 音乐与资源管理 .....      | 5  |
| 六、功能测试 .....          | 6  |
| 七、打包部署与仓库提交 .....     | 11 |
| 八、AI 工具使用说明 .....     | 12 |
| 九、开发中遇到的问题与解决方法 ..... | 12 |
| 十、收获与总结 .....         | 13 |

# 高级语言程序设计大作业实验报告

## 一、作业题目

本次大作业题目为“顶流争夺保卫战”。本项目为一个图形化塔防小游戏，玩家需要通过建造、升级和出售防御塔来阻止敌人沿固定路径到达终点。

## 二、开发软件与运行环境

本项目使用 C++语言和 Qt Widgets 框架完成，使用 CMake 管理项目构建。

| 项目     | 内容                                                    |
|--------|-------------------------------------------------------|
| 开发语言   | C++                                                   |
| 图形界面框架 | Qt Widgets                                            |
| 开发工具   | Qt Creator                                            |
| 构建工具   | CMake/Ninja                                           |
| 测试环境   | Windows 11, Qt 6.10.2, Desktop Qt 6.10.2 MinGW 64-bit |
| 运行依赖   | Qt Widgets 模块、Qt Multimedia 模块                        |

## 三、课题要求

本项目按照大作业要求，完成一个具有图形界面的 C++小程序。项目要求包括能够体现一定创新性和完整性，提交 GitHub 或 Gitee 仓库，提交项目报告，并录制项目讲解视频。

本项目还提供了 Release 发行版压缩包，方便在不配置 Qt 开发环境的情况下直接运行。

## 四、项目总体设计

### 1. 整体流程

程序启动后，首先进入开始界面。玩家可以选择开始游戏、查看游戏说明或退出游戏。点击开始游戏后，主窗口切换到游戏主界面，游戏循环开始运行。GameWidget 通过 QTimer 定时调用 updateGame()，每次调用都会更新敌人、塔、子弹和游戏状态。

| 序号 | 流程   | 说明                                    |
|----|------|---------------------------------------|
| 1  | 启动程序 | 创建 QApplication 和 MainWindow，显示进入界面。  |
| 2  | 进入菜单 | StartMenuWidget 显示开始界面，并处理开始、说明、退出按钮。 |
| 3  | 开始游戏 | MainWindow 切换到 GameWidget，开始战斗音乐。     |

|   |       |                                    |
|---|-------|------------------------------------|
| 4 | 生成敌人  | GameWidget 根据波次队列生成 Enemy。         |
| 5 | 游戏循环  | QTimer 周期性调用 updateGame(), 更新游戏对象。 |
| 6 | 防御塔攻击 | Tower 检测范围内敌人并生成 Bullet。           |
| 7 | 子弹命中  | Bullet 追踪目标, 命中后扣除敌人血量。            |
| 8 | 胜负判断  | 生命值为 0 则失败; 敌人全部清空则胜利。             |

## 2. 类与模块划分

| 模块/类            | 主要职责      | 说明                                   |
|-----------------|-----------|--------------------------------------|
| MainWindow      | 页面切换、音乐控制 | 管理开始界面和游戏界面, 控制菜单音乐、战斗音乐和最终波次音乐。     |
| StartMenuWidget | 开始界面和说明界面 | 显示进入界面图片, 处理“开始游戏”“游戏说明”“退出游戏”等点击区域。 |
| GameWidget      | 游戏主场景     | 负责地图绘制、游戏循环、建塔、波次、胜负判断和结果界面。         |
| Enemy           | 敌人对象      | 负责敌人属性、路径移动、受伤、死亡判断和特殊能力。            |
| Tower           | 防御塔对象     | 负责塔的属性、攻击范围、攻击间隔、升级和出售。              |
| Bullet          | 子弹对象      | 负责子弹追踪目标敌人、移动和命中判定。                  |
| resources.qrc   | 资源管理      | 统一管理图片、音乐等资源文件。                      |

## 3. 游戏循环设计

本项目没有使用 while 死循环, 而是使用 Qt 的 QTimer 定时器。定时器每隔约 30 ms 调用一次 updateGame(), 相当于游戏主循环。每一帧中, 程序会依次执行出怪、敌人移动、防御塔攻击、子弹移动、对象清理和胜负判断。

# 五、主要模块设计与实现

## 1. 开始界面与说明界面

StartMenuWidget 使用整张图片作为开始界面和游戏说明界面, 避免大量控件拼接

导致风格不统一。按钮功能通过 QRect 设置点击区域实现，包括开始游戏、游戏说明、退出游戏以及说明页返回主菜单。

## 2. 地图与路径设计

主游戏界面使用一张森林地图作为背景。敌人的实际移动路线由 `initPaths()` 中的 `QPointF` 坐标点定义，而不是自动识别图片道路。项目设置了两条敌人路线，敌人生成时轮流选择不同路线，从而形成多路径塔防效果。

建塔时，程序会计算点击位置到路径线段的距离，如果距离道路太近，则禁止建塔。这样可以防止玩家把防御塔建在道路上。

## 3. 敌人系统

`Enemy` 类保存敌人的类型、生命值、速度、护甲、奖励金币、当前位置和移动路径。敌人每帧朝当前目标路径点移动，到达后切换到下一个路径点。

项目中包含 Bug、DDL、Virus、Boss 等敌人类型。DDL 速度较快，Virus 血量较高，Boss 出现在最终波次。后期波次还会对敌人进行强化，提高游戏难度。

## 4. 防御塔系统

`Tower` 类负责防御塔的属性、攻击范围、攻击间隔、升级和出售。游戏中有普通塔、速射塔和重炮塔三种防御塔。普通塔属性均衡，速射塔攻击频率高，重炮塔单次伤害高。

当玩家点击地图时，`GameWidget` 会判断是否点中已有塔。如果没有点中已有塔，则尝试在点击位置建造当前选中的防御塔。按键 1、2、3 可切换塔型，U 键升级选中的塔，X 或 Delete 键出售选中的塔。

## 5. 子弹系统

`Bullet` 类负责追踪目标敌人并造成伤害。防御塔攻击时会生成子弹对象，子弹每帧朝目标敌人当前位置移动。当子弹接近敌人时，调用敌人的 `takeDamage()` 扣血并将子弹标记为结束。

为了避免访问已删除对象，若目标敌人已经死亡、到达终点或为空，子弹会自动失效。

## 6. 胜负判断与结果界面

`checkGameResult()` 负责胜负判断。生命值小于等于 0 时游戏失败；所有敌人生成完毕且场上敌人清空时游戏胜利。游戏结束后停止定时器，并绘制对应胜利或失败图片。

胜利和失败界面提供“重新挑战”和“返回主页”两个功能。重新挑战会清空敌人、塔和子弹，并重置金币、生命、波次等状态；返回主页通过信号通知 `MainWindow` 切换回开始界面。

## 7. 音乐与资源管理

项目使用 `resources.qrc` 管理图片和音乐资源。界面图片、地图图片、敌人图片、胜利失败图片以及背景音乐均加入 Qt 资源系统。背景音乐使用 `QMediaPlayer` 和

QAudioOutput 循环播放，一次性音效使用 QSoundEffect 播放。

## 六、功能测试

本项目的功能验证主要围绕界面切换、游戏玩法、波次推进、胜负判断、音乐播放和发行版运行进行。以下测试均在本机环境中完成，并对部分关键功能进行了多次运行验证。

### 1. 界面与资源显示验证

| 序号 | 测试内容   | 测试方法       | 预期/实际结果              | 结论 |
|----|--------|------------|----------------------|----|
| 1  | 进入界面显示 | 启动程序。      | 显示进入界面图片，窗口比例正常。     | 通过 |
| 2  | 游戏说明界面 | 点击“游戏说明”。  | 显示说明界面图片。            | 通过 |
| 3  | 说明界面返回 | 点击“返回主菜单”。 | 返回进入界面。              | 通过 |
| 4  | 开始游戏   | 点击“开始游戏”。  | 进入主游戏界面。             | 通过 |
| 5  | 退出游戏   | 点击“退出游戏”。  | 程序正常关闭。              | 通过 |
| 6  | 主地图显示  | 进入游戏后观察地图。 | 显示地图图片，不再显示默认网格和旧道路。 | 通过 |
| 7  | 胜负界面图片 | 触发胜利或失败。   | 显示对应胜利/失败图片。         | 通过 |

### 2. 建塔与战斗功能验证

| 序号 | 测试内容   | 测试方法              | 预期/实际结果           | 结论 |
|----|--------|-------------------|-------------------|----|
| 1  | 普通塔建造  | 按 1 后点击可建塔区域。     | 金币足够时成功建造普通塔。     | 通过 |
| 2  | 速射塔建造  | 按 2 后点击可建塔区域。     | 金币足够时成功建造速射塔。     | 通过 |
| 3  | 重炮塔建造  | 按 3 后点击可建塔区域。     | 金币足够时成功建造重炮塔。     | 通过 |
| 4  | 禁止道路建塔 | 点击道路或道路附近。        | 不能建塔，避免塔建在路上。     | 通过 |
| 5  | 金币不足处理 | 金币不足时尝试建塔。        | 不能建塔，并给出提示。       | 通过 |
| 6  | 选择已有塔  | 点击已经建造的塔。         | 该塔被选中。            | 通过 |
| 7  | 防御塔升级  | 选中塔后按 U。          | 金币足够时等级提升，攻击能力增强。 | 通过 |
| 8  | 防御塔出售  | 选中塔后按 X 或 Delete。 | 塔被删除，并返还部分金币。     | 通过 |

|    |       |           |              |    |
|----|-------|-----------|--------------|----|
| 9  | 塔攻击敌人 | 敌人进入攻击范围。 | 塔生成子弹并攻击敌人。  | 通过 |
| 10 | 子弹命中  | 观察子弹追踪敌人。 | 子弹命中后敌人血量减少。 | 通过 |

3. 敌人、波次与结果验证

| 序号 | 测试内容   | 测试方法            | 预期/实际结果             | 结论 |
|----|--------|-----------------|---------------------|----|
| 1  | 敌人路径移动 | 等待敌人生成并观察移动。    | 敌人沿两条路径移动，基本位于道路中心。 | 通过 |
| 2  | 敌人死亡奖励 | 击败敌人。           | 敌人消失，金币增加。          | 通过 |
| 3  | 敌人到达终点 | 让敌人未被击败到达终点。    | 生命值减少。              | 通过 |
| 4  | 波次推进   | 当前波敌人清空后继续游戏。   | 进入下一波，波次显示更新。       | 通过 |
| 5  | 最终波次音乐 | 进入第六波。          | 切换最终波次音乐，并播放一次音效。   | 通过 |
| 6  | 失败判断   | 生命值降为 0。        | 停止游戏并显示失败界面。        | 通过 |
| 7  | 胜利判断   | 所有敌人被消灭。        | 停止游戏并显示胜利界面。        | 通过 |
| 8  | 重新挑战   | 在结果界面点击重新挑战。    | 游戏状态重置，重新开始。        | 通过 |
| 9  | 返回主页   | 在结果界面点击返回主页。    | 返回进入界面，游戏重置。        | 通过 |
| 10 | 发行版运行  | 打包后双击 final.exe | 解压完整压缩包后可直接运行。      | 通过 |

4. 主要功能介绍（详细说明）

（1）开始界面与页面切换测试

测试过程与输入：运行程序后，首先进入 StartMenuWidget 开始界面，分别点击“开始游戏”“游戏说明”“退出游戏”三个区域；进入游戏说明界面后，再点击图片中的

“返回主菜单”按钮。

预期与实际结果：开始界面由 `StartMenuWidget::paintEvent()` 绘制背景图片，通过 `mousePressEvent()` 判断鼠标点击位置。当点击“开始游戏”区域时，程序发出 `startGameClicked()` 信号，`MainWindow` 接收信号后通过 `QStackedWidget` 切换到 `GameWidget` 游戏界面。点击“游戏说明”后，`m_showHelp` 被设置为 `true`，界面切换为说明图片；点击“返回主菜单”后，`m_showHelp` 重新变为 `false`。点击“退出游戏”后，程序正常关闭。实际运行结果与预期一致，页面切换过程正常。

测试通过。

## (2) 主游戏地图与路径对齐测试

测试过程与输入：点击“开始游戏”进入 `GameWidget`，观察主游戏地图背景、起点终点标记和敌人的移动路径。开发过程中曾将 `m_debugDrawPath` 设置为 `true`，显示代码路径和网格，用于对齐背景图中的道路。

预期与实际结果：主地图背景由 `GameWidget::drawBackground()` 绘制，路径坐标在 `initPaths()` 中初始化，并保存到 `m_paths` 中。敌人对象生成时会获得其中一条路径，并按照路径点依次移动。实际测试中，敌人能够从两条路线的起点生成，并沿背景图中的道路向终点移动。经过多次调整路径点坐标后，敌人路线与地图道路基本重合，没有明显偏离道路的情况。正式运行时关闭调试路径，仅保留起点和终点标记。

测试通过。

## (3) 建塔与塔型切换测试

测试过程与输入：在主游戏界面中，分别按下数字键 1、2、3 切换当前塔型，并在地图空地处点击鼠标左键尝试建塔。

预期与实际结果：键盘事件由 `GameWidget::keyPressEvent()` 处理，按下 1、2、3 后分别修改 `m_selectedTowerType`，对应普通塔、速射塔和重炮塔。鼠标点击建塔时，`mousePressEvent()` 会根据当前位置调用 `canBuildTowerAt()` 判断是否可以建塔，包括金币是否足够、是否靠近道路、是否与已有塔过近等条件。实际测试中，三种塔均能正确切换和建造，建塔后金币减少，防御塔被加入 `m_towers` 容器中。若点击道路附近或已有塔附近，程序不会建塔，避免出现塔与路径或其他塔重叠的情况。

测试通过。

## (4) 防御塔升级与出售测试

测试过程与输入：点击已经建造的防御塔，使其成为当前选中塔，然后按下 `U` 键进行升级；再次选中防御塔后，按 `X` 或 `Delete` 键进行出售。

预期与实际结果：鼠标点击已有防御塔时，程序通过 `Tower::containsPoint()` 判断是否选中该塔，并将其保存为 `m_selectedTower`。按下 `U` 后调用 `upgradeSelectedTower()`，



该函数进一步调用 `Tower::canUpgrade()` 和 `Tower::upgradeCost()` 判断能否升级，并在金币足够时调用 `Tower::upgrade()` 修改塔的等级、伤害、范围和攻击间隔。按下 X 或 Delete 后调用 `sellSelectedTower()`，移除选中塔并返还部分金币。实际测试中，升级后防御塔等级和攻击效果发生变化；出售后防御塔从场景中消失，金币数增加。

测试通过。

#### **(5) 敌人生成与波次推进测试**

测试过程与输入：正常运行游戏，观察不同波次中敌人的生成情况，重点检查普通敌人、快速敌人、血量较高敌人以及 Boss 是否按设定出现。

预期与实际结果：敌人波次由 `initSpawnQueue()` 初始化，使用字符串表示不同敌人类型，例如“bug”、“ddl”、“virus”和“boss”。在 `updateGame()` 中，程序根据当前波次索引、生成间隔和已生成数量调用 `spawnEnemy()` 创建敌人对象，并加入 `m_enemies` 容器。实际测试中，前期主要生成普通敌人，后续波次逐渐出现 DDL、Virus 等类型，最终波次能够生成 Boss。随着 `m_currentWaveIndex` 推进，敌人数量和压力逐渐增加，波次推进逻辑正常。

测试通过。

#### **(6) 敌人移动与特殊能力测试**

测试过程与输入：观察不同敌人在路径中的移动情况，重点测试 DDL 的快速移动、Virus 的高血量表现以及 Boss 的最终波次表现。

预期与实际结果：敌人移动逻辑主要由 `Enemy::update()` 实现。敌人每一帧根据当前位置和当前目标路径点计算移动方向，到达一个路径点后切换到下一个路径点。不同类型敌人的属性在 `Enemy::setupByType()` 中设置。实际测试中，普通敌人能够稳定沿路径移动；DDL 移动速度较快，能体现快速敌人的特点；Virus 血量较高，需要更多攻击才能消灭；Boss 在最终波次出现，血量和威胁明显高于普通敌人。敌人到达终点后能够被正确识别。

测试通过。

#### **(7) 防御塔攻击与子弹追踪测试**

测试过程与输入：在敌人路线附近建造防御塔，观察敌人进入攻击范围后防御塔是否自动攻击，子弹是否能够追踪敌人并造成伤害。

预期与实际结果：防御塔攻击逻辑由 `Tower::updateAttack()` 实现。每座塔通过攻击计数器控制攻击频率，当计数器达到攻击间隔后，遍历 `m_enemies`，寻找攻击范围内的目标敌人。若找到目标，则创建 `Bullet` 对象并加入 `m_bullets`。子弹在 `Bullet::update()` 中持续向目标敌人移动，接近目标后调用敌人的受伤函数扣除血量。实际测试中，敌人进入攻击范围后防御塔能够自动发射子弹，子弹能够追踪目标敌人，

命中后敌人血条减少。敌人死亡后从地图上移除，玩家金币增加。

测试通过。

#### **(8) 敌人死亡、金币奖励与生命扣除测试**

测试过程与输入：正常进行游戏，观察敌人被击杀和敌人到达终点两种情况，检查金币和生命值变化。

预期与实际结果：在 `updateGame()` 中，程序会遍历 `m_enemies` 判断敌人状态。如果敌人死亡，则根据其奖励值增加 `m_gold`，并从敌人容器中移除；如果敌人到达终点，则根据敌人造成的伤害减少 `m_life`。删除敌人前，程序会处理与该敌人相关的子弹，避免子弹继续访问已经失效的目标。实际测试中，击杀敌人后金币增加，敌人到达终点后生命值减少，游戏状态显示栏能够同步更新。

测试通过。

#### **(9) 胜利与失败结算测试**

测试过程与输入：分别测试失败和胜利两种情况。失败测试中，故意少建塔或不建塔，使敌人到达终点并消耗生命值；胜利测试中，通过合理建塔和升级消灭所有波次敌人。

预期与实际结果：胜负判断由 `checkGameResult()` 完成。当 `m_life <= 0` 时，设置 `m_gameFinished=true`，并将 `m_gameWon` 设为失败状态；当所有敌人生成完毕且 `m_enemies` 为空时，设置 `m_gameFinished=true`，并将 `m_gameWon` 设为胜利状态。游戏结束后，计时器停止，`paintEvent()` 中调用结果界面绘制函数显示胜利或失败图片。实际测试中，失败时能够显示失败界面，胜利时能够显示胜利界面；两个界面中的“重新挑战”和“返回主页”按钮均能正常使用。

测试通过。

#### **(10) 重新挑战与返回主页测试**

测试过程与输入：在胜利或失败界面中，分别点击“重新挑战”和“返回主页”按钮。

预期与实际结果：游戏结束界面的鼠标点击仍由 `GameWidget::mousePressEvent()` 处理。若点击“重新挑战”，调用 `restartGame()`，该函数会清空敌人、防御塔和子弹，重置金币、生命、波次、Boss 状态、生成计数器等变量，并重新初始化路径和波次。若点击“返回主页”，`GameWidget` 发出 `backToMenuRequested()` 信号，`MainWindow` 接收后切换回 `StartMenuWidget`。实际测试中，重新挑战后游戏能够从初始状态重新开始；返回主页后能够回到开始界面，并可再次进入游戏。

测试通过。

#### **(11) 音乐播放与切换测试**

测试过程与输入：运行程序后观察开始界面音乐，点击开始游戏后观察战斗音乐，进入最终波次后观察最终波次音乐切换，返回主页后观察音乐是否恢复。

预期与实际结果：音乐播放由 MainWindow 统一控制，背景音乐使用 QMediaPlayer 和 QAudioOutput 播放，短音效使用 QSoundEffect 播放。程序启动时播放开始界面音乐；点击开始游戏后切换为战斗音乐；进入第六波时，GameWidget 发出 finalWaveStarted() 信号，MainWindow 接收后切换为最终波次音乐；返回主页后重新播放开始界面音乐。实际测试中，音乐能够按界面和波次状态正常切换。

测试通过。

### **(12) 资源文件加载测试**

测试过程与输入：运行程序后观察开始界面、说明界面、主地图、敌人图片、胜利失败图片和音乐是否正常加载。

预期与实际结果：项目中的图片和音乐通过 resources.qrc 管理，代码中使用 Qt 资源路径读取对应文件。实际测试中，开始界面、说明界面、主游戏地图、敌人图片、胜利失败界面均能够正常显示，背景音乐和战斗音乐能够正常播放。测试过程中曾出现资源加载失败的问题，原因是资源文件中的路径与代码中的读取路径不一致。统一文件名和资源路径后，资源加载恢复正常。

测试通过。

### **(13) 发行版运行测试**

测试过程与输入：将项目切换到 Release 模式构建，找到生成的 final.exe，使用 windeployqt 对程序进行部署，将生成的可运行文件夹压缩为 final\_release\_windows.zip，解压后直接双击 final.exe 运行。

预期与实际结果：在不打开 Qt Creator 的情况下，双击发行版中的 final.exe 应能够直接运行游戏，且图片、音乐和基本游戏功能正常。实际测试中，直接运行发行版可以正常进入开始界面，开始游戏、说明界面、主游戏界面、敌人图片、音乐和胜负界面均能正常使用。测试过程中曾出现只复制 final.exe 导致缺少 Qt6Gui.dll 的问题，后续通过 windeployqt 自动生成 Qt 运行库和插件文件夹后解决。

测试通过。

综上，本项目的主要功能均通过运行验证。游戏能够完成从开始界面进入战斗、建塔攻击、波次推进、胜负结算和返回主页的完整流程，发行版也能够在不打开 Qt Creator 的情况下直接运行。

## **七、打包部署与仓库提交**

本项目已上传到 GitHub 和 Gitee 仓库，并保留了多次提交记录，用于体现开发过程中的版本迭代。仓库中包含完整源码、资源文件和 README 说明。

为了方便直接运行，本项目还使用 Release 模式构建程序，并通过 windeployqt 生成可运行文件夹，将其压缩为 final\_release\_windows.zip。使用者下载完整压缩包并解压后，可以直接双击 final.exe 运行。

项目仓库链接：

GitHub: <https://github.com/yinwenjie521yeah-crypto/26C->

Gitee: [https://gitee.com/Yin\\_wen-jie/26-c](https://gitee.com/Yin_wen-jie/26-c)

B 站讲解视频链接：

**【南开大学 26C++】用 Qt/C++ 做了个萌系塔防，结果小怪全是顶流表情包？！（要脸，别赞）**

[https://www.bilibili.com/video/BV1BmRJBWEqL?vd\\_source=9c3607711c0a7d4a3a69d4856fcaee59](https://www.bilibili.com/video/BV1BmRJBWEqL?vd_source=9c3607711c0a7d4a3a69d4856fcaee59)

### 八、AI 工具使用说明

本项目开发过程中使用了 AI 工具进行辅助，具体如下。

| AI 工具          | 版本/类型           | 用途                       | 使用位置                              |
|----------------|-----------------|--------------------------|-----------------------------------|
| ChatGPT        | GPT-5.5Thinking | 辅助整理开发思路、调试思路、README     | 项目文档、报错分析、代码修改步骤说明。               |
| ChatGPT 图像生成工具 | OpenAI 图像生成     | 辅助生成游戏美术资源，提高界面完整性和视觉效果。 | 开始界面、游戏说明界面、主地图背景、胜利/失败界面及部分角色素材。 |

需要说明的是，AI 主要用于美术素材生成、代码整理和调试思路辅助。游戏核心功能逻辑由本人使用 C++ 和 Qt 实现，包括页面切换、敌人路径移动、防御塔攻击、子弹追踪、波次控制、胜负判断、音乐切换和结果界面交互等。

### 九、开发中遇到的问题与解决方法

| 问题                 | 原因分析                                                                        | 解决方法                                                    |
|--------------------|-----------------------------------------------------------------------------|---------------------------------------------------------|
| 图片资源加载失败           | resources.qrc 中前缀为 /images，同时文件位于 images 文件夹下，实际路径需要写成 :/images/images/xxx。 | 统一资源路径，检查 qrc 中的文件名和代码中的加载路径。                           |
| 敌人后期图片变回文字         | 后期强化逻辑尝试加载装甲版图片，但没有准备装甲素材。                                                  | 取消装甲版图片切换，强化仅改变属性，继续使用普通敌人图片。                           |
| Qt Multimedia 报错   | CMakeLists.txt 未链接 Multimedia 模块或环境未安装该模块。                                  | 在 CMake 中加入 Widgets Multimedia，并确认 Qt 安装包包含 Multimedia。 |
| GitHub/Gitee 下载后他人 | 下载源码需要 Qt 环境，不                                                              | 删除.qtcrc 和                                              |

|                 |                                |                                           |
|-----------------|--------------------------------|-------------------------------------------|
| 无法运行            | 能直接双击运行；Qt Creator 本地配置文件曾被提交。 | *.user，完善 README，并提供 windeployqt 打包的发行版。  |
| 直接运行 exe 缺少 dll | 只复制了 final.exe，未复制 Qt 依赖库。     | 使用 windeployqt 部署，将 exe、dll 和插件文件夹一同压缩发布。 |

## 十、收获与总结

通过本次大作业，我进一步熟悉了 Qt Widgets 的事件处理、绘图机制、资源管理和 CMake 构建流程。项目中使用 QTimer 实现游戏循环，使用多个类分别管理敌人、防御塔、子弹和游戏主场景，使程序结构更加清晰。

在开发过程中，我对图形界面程序中的资源路径、对象生命周期、信号与槽、模块链接和部署打包有了更深入的理解。尤其是 windeployqt 打包过程让我认识到源码运行和可执行程序发布之间的区别。

本项目还让我体会到调试和版本管理的重要性。通过 GitHub/Gitee 多次提交，可以记录开发过程中的不同版本；通过 README 和实验报告，可以让他人更清楚地理解项目的运行环境、功能设计和使用方式。